

NumPy Arrays in One Dimension

Array creation and dtypes, elementwise operations, reductions, slicing and views, selection by index array and by boolean mask, assignment through an index, and locating matches. All examples are one-dimensional.

01 Why an array and not a list	06 Index arrays and boolean masks
02 Making an array, and reading it	07 Writing through an index
03 Elementwise operations	08 Finding the first match
04 Reductions	09 Loop to expression
05 Slicing, and what a slice is	

Scope. A Python list stores references to objects scattered across memory, and every operation on it runs in the interpreter one element at a time. A NumPy array stores raw values of a single type in one contiguous block, and operations on it run in compiled code that does not return to the interpreter between elements. Three consequences of that difference are covered below: the speed of array operations, the fixed element type, and the fact that a slice does not copy.

After reading it you should be able to do the following without looking anything up.

- Create arrays with `np.array`, `np.zeros`, `np.ones`, `np.arange` and `np.linspace`, and read `shape`, `dtype`, `ndim` and `size`.
- Apply arithmetic and comparison to a whole array without writing a loop.
- Collapse an array to a single number with `sum`, `mean`, `min` and `max`.
- Select elements with a slice, with an array of indices, and with a boolean mask, and say which of those copies and which does not.
- Write into an array through any of those three, and find the first element that satisfies a condition.

Every example assumes `import numpy as np`.

01

Why an array and not a list

A list and an array holding the same million integers differ in memory layout, total size and per-element access cost.

TWO MEMORY PICTURES

A list owns a contiguous block of references. The integers themselves are separate Python objects elsewhere in memory, each with its own header. On this machine a small `int` costs 28 bytes, so a list of a million of them is about 8 MB of references plus roughly 28 MB of objects.

An array owns a contiguous block of values. A million `int64` values are 8 bytes each and nothing else: 8 MB in total, packed end to end, with no per-element object.

That has three consequences. The data occupies about a quarter of the memory. Reading an element is one memory access rather than two. And because the values are adjacent, the processor's cache and its vector instructions can operate on many of them at a time.

It costs two things, and neither is negotiable.

- **The type is fixed at creation.** Every element occupies the same number of bytes and is interpreted the same way, so an array cannot hold an integer next to a string the way a list can. That type is the array's `dtype`.
- **The size is fixed at creation.** There is no `append`. Growing an array means allocating a new one and copying, so arrays are built once at their final size rather than accumulated element by element.

WHAT THAT IS WORTH

Summing the squares of n numbers, as a Python loop over a list and as an array expression. Best of nine runs, on Apple silicon with Python 3.14 and NumPy 2.5:

N	SUM(X * X FOR X IN LST)	(ARR * ARR).SUM()	RATIO
100 000	0.0031 s	0.000048 s	65×
1 000 000	0.0340 s	0.00082 s	41×
10 000 000	0.4050 s	0.0106 s	38×

Read the ratio column carefully. Both versions are $O(n)$ and remain $O(n)$: each row's times are roughly ten times the row above it. What NumPy removes is the per-element interpreter overhead, which is a large constant factor, not a change of complexity.

The factor is not constant either. It falls from 65× to about 38× as the data stops fitting in cache and both versions become more limited by memory access than by arithmetic. Exact figures depend on the machine, the Python version and the NumPy build, so treat the magnitude as the point and not the digits.

The practical consequence is that NumPy will not rescue a quadratic algorithm. It makes a linear pass cheaper by a large constant.

02

Making an array, and reading it

Arrays are constructed from existing data, or from a shape and a fill value. Four attributes describe the result: `shape`, `dtype`, `ndim` and `size`.

```
a = np.array([2, 3, 4, 1, 5], dtype=np.int32)

a.shape      # (5,)      length along each axis, as a tuple
a.dtype      # dtype('int32') the type of every element
a.ndim       # 1        number of axes, same as len(a.shape)
a.size       # 5        total number of elements, the product of shape
a.itemsize   # 4        bytes per element
len(a)       # 5        length along axis 0
```

The nesting depth of the lists you pass decides the number of axes, and every list at the same depth must be the same length. A ragged one is an error, not a jagged array:

```
np.array([[1, 2], [3]])
# ValueError: setting an array element with a sequence. The requested array
# has an inhomogeneous shape after 1 dimensions.
```

BUILDING FROM A SHAPE

```
np.zeros(5)           # [0. 0. 0. 0. 0.]   dtype float64 by default
np.ones(3, dtype=np.int32) # [1 1 1]
np.full(4, 7)         # [7 7 7 7]
np.zeros_like(a)       # zeros with a's shape AND a's dtype
np.arange(5)           # [0 1 2 3 4]     like range: stop is excluded
np.arange(2, 10, 3)    # [2 5 8]
np.arange(10, 2, -3)   # [10 7 4]      negative step walks backwards
np.arange(2, 10, -2)   # []             start below stop, negative step
np.linspace(0, 1, 5)   # [0. 0.25 0.5 0.75 1.] endpoint INCLUDED
```

The difference between the last two matters. `np.arange` takes a step and excludes the stop, like `range`. `np.linspace` takes a count and includes both ends, which is what you want for sampling an interval evenly. Unlike `range`, `np.arange` is not lazy: it allocates every value immediately.

DTYPE, AND WHEN TO STATE IT

If you do not name a dtype, NumPy infers one: integer input gives `int64`, anything containing a float gives `float64`, and `np.zeros` and `np.ones` give `float64`. `astype` makes a new array with a different type, copying as it converts:

```
a = np.array([-2, 3, -4], dtype=np.int32)
np.fabs(a)           # [2. 3. 4.]   float64: fabs always returns floats
np.fabs(a).astype(np.int32) # [2 3 4]   back to integers, as a new array
```

A NumPy dtype is not the same thing as a Python type. Every Python object has a type, and the type of a NumPy array is `ndarray`; the dtype describes the elements inside it.

INTEGER ARITHMETIC WRAPS AROUND, SILENTLY

A Python `int` grows as large as it needs to. A NumPy integer has a fixed width, and exceeding it wraps rather than raising. Nothing is printed and nothing in the output looks unusual except the number.

```
a = np.array([2_000_000_000, 2_000_000_000], dtype=np.int32)
a + a           # [-294967296 -294967296]

b = np.array([100, 200], dtype=np.uint8)
b * 2           # [200 144]      144 is 400 - 256
```

Building an array is the one place NumPy checks: `np.array([400], dtype=np.uint8)` raises `OverflowError`. Converting does not, so `np.array([400]).astype(np.uint8)` gives 144. Widen before you compute, not after: if a squared or accumulated value might leave the range, call `arr.astype(np.int64)` first.

Elementwise operations

NumPy defines the arithmetic and comparison operators over whole arrays, applying them elementwise in compiled code.

Adding two arrays of the same shape, written out by hand, is a loop:

```

out = np.zeros_like(arr1)
for i in range(arr1.shape[0]):
    out[i] = arr1[i] + arr2[i]

out = arr1 + arr2          # the same thing, in compiled code
out = np.add(arr1, arr2)   # + is the operator form of np.add

```

Every operator has a named function behind it, and they are interchangeable: `-` is `np.subtract`, `*` is `np.multiply`, `/` is `np.true_divide`, `//` is `np.floor_divide`, `%` is `np.mod`, `**` is `np.power`. As in plain Python, `/` always produces floats and `//` floors towards negative infinity.

Comparisons work the same way and produce a boolean array rather than a single `True` :

```

arr = np.array([2, 3, 4, 1])
arr > 2          # [False  True  True False]
arr % 2 == 0     # [ True False  True False]
(arr > 2).sum()  # 2      True counts as 1, so this counts matches

```

ONE OPERAND MAY BE A SCALAR

`arr + 42` adds 42 to every element. The scalar is treated as though it had been repeated to the array's shape, without actually being copied. This is the simplest case of broadcasting; combining two arrays of *different* shapes follows rules that are week 4's material. Two 1D arrays of different lengths do not combine:

```

np.array([1, 2]) + np.array([1, 2, 3])
# ValueError: operands could not be broadcast together with shapes (2,) (3,)

```

ELEMENTWISE MAXIMUM AGAINST THE MAXIMUM

Two similarly named functions that do different jobs, and are easily confused:

```

x = np.array([2, 3, 4, 1, 5])
np.maximum(x, 3)    # elementwise: [3 3 4 3 5]      same shape as x
np.amax(x)          # collapse:      5              a single value
x.max()             # collapse:      5              the method form of np.amax

```

`np.maximum` and `np.minimum` compare two things elementwise and return an array. `np.amax` and `np.amin` collapse one array to its largest or smallest value. `np.maximum(arr, 0)` is the standard way to clamp negatives to zero.

WRITING THE RESULT IN PLACE

Every elementwise operation allocates a new array for its result. When the array is large and the old values are not needed, `+=` or the `out=` argument write into existing memory instead:

```

arr += 2          # in place, no new array
np.add(arr, 2, out=arr) # the same thing, written out
np.maximum(arr, 0, out=arr) # out= works for functions with no operator form

```

Reductions

A reduction takes an array and returns fewer numbers than it was given. On a one-dimensional array it returns a single value.

```
m = np.array([2, 3, 4, 1, 5, 8])
m.sum()          # 23          np.sum(m)
m.prod()         # 960         np.prod(m)
m.mean()         # 3.8333...   np.mean(m), always a float
m.min()          # 1
m.max()          # 8
m.std()          # standard deviation, np.std(m)
m.var()          # variance, np.var(m)
m.argmax()       # 5          the POSITION of the largest value
m.argmin()       # 3          the position of the smallest
```

Ties in `argmax` and `argmin` are broken by position: the first occurrence wins.

Reductions compose with elementwise work, and that pairing covers a large fraction of everyday array code: transform every element, then collapse.

```
((values - 100) ** 2).mean()      # mean squared deviation from 100
(scores >= 50).sum()             # how many passed
(scores >= 50).mean()            # what fraction passed
np.cumsum(values)                # a running total, one entry per input
```

`np.cumsum` is not a reduction: it returns an array the same length as its input, where entry *i* is the total of everything up to and including *i*. It is the array form of an accumulator loop.

THE ACCUMULATOR IS WIDER THAN THE ELEMENTS

`sum` on an `int32` array accumulates in `int64`, so the total of many large `int32` values does not overflow. The elementwise step before it has no such protection: in `(arr * arr).sum()` the multiplication happens first, at the array's own dtype, and can wrap before the sum ever sees it.

A reduction can also be applied along one axis of a multidimensional array, with the `axis=` argument. That is week 4's material and needs a second axis to mean anything.

05

Slicing, and what a slice is

Slice syntax is identical to Python lists. The result is not: a slice of an array shares the original's memory.

```
a = np.arange(10)          # [0 1 2 3 4 5 6 7 8 9]
a[2:7]                     # [2 3 4 5 6]   start:stop, stop excluded
a[2:7:2]                   # [2 4 6]       start:stop:step
a[2:]                     # from 2 to the end
a[:-2]                    # everything but the last two
a[::-1]                   # [9 8 7 6 5 4 3 2 1 0] reversed
a[7:2:-2]                 # [7 5 3]       a negative step needs start above stop
a[2:7:-2]                 # []           otherwise nothing is selected
```

One behaviour has no equivalent in single-element indexing. An out-of-range index raises; an out-of-range slice is clipped:

```

a[12:]      # []                                no error
a[:12]      # [0 1 2 3 4 5 6 7 8 9]            no error
a[12]       # IndexError: index 12 is out of bounds for axis 0 with size 10

```

This is convenient and it hides mistakes. A window computed slightly wrong returns a short array rather than complaining, and the error surfaces later as a wrong number.

A SLICE IS A VIEW

Slicing a list copies. Slicing an array does not.

```

my_list = list(range(10))
part = my_list[2:7]
part[0] = 99
my_list[2]          # 2      the list is untouched: slicing copied

a = np.arange(10)
part = a[2:7]
part[0] = 99
a[2]                # 99     the array changed: slicing did NOT copy

```

A basic slice allocates no data. It is a second `ndarray` object pointing into the same memory, differing only in where it starts, how many elements it covers, and how far apart they are. Writing through either name changes the one block of memory both of them describe.

This has a practical benefit: selecting a region of a large array costs nothing, so slicing inside a loop does not allocate the way slicing a list does.

WHEN YOU NEED A COPY, ASK FOR ONE

If a function is going to modify what it was handed, or you want to keep a piece of an array after the original changes, take an explicit copy. It is the only thing that breaks the link.

```

part = a[2:7].copy()      # or np.copy(a[2:7])
part[0] = -1
a[2]                      # unchanged

```

The reverse mistake is equally common: copying out of habit, in a loop, over a large array, and paying for memory and time that the view would have avoided.

06

Index arrays and boolean masks

Selection by an array of positions, and selection by a condition. Neither is expressible as a slice, and both return copies.

SELECTING BY AN ARRAY OF INDICES

Indexing with a list or array of integers returns the elements at those positions, in that order. Positions may repeat and may be negative, and the result has the length of the index array, not of the original:

```

a = np.array([0, 10, 20, 30, 40])
a[[2, 3]]          # [20 30]
a[[-2, 0, 1, 0]]   # [30  0 10  0]    repeats are allowed
a[[]]              # []              shape (0,)
a[np.array([4, 4, 4])] # [40 40 40]

```

Unlike a slice, this returns a **copy**. The positions are arbitrary, so there is no start, length and step that could describe them as a view.

SELECTING BY A BOOLEAN MASK

A comparison produces a boolean array the same length as the original. Indexing with that array keeps the elements where it is `True` :

```

a = np.array([0, 10, 20, 30, 40])
mask = a % 30 == 0
mask          # [ True False False  True False]
mask.dtype    # bool
a[mask]       # [ 0 30]
a[a > 15]     # [20 30 40]    written in one step

```

A mask used for indexing must be the same length as the axis it indexes. This also returns a copy.

COMBINING CONDITIONS

Conditions combine with `&` for and, `|` for or, and `~` for not. Python's `and`, `or` and `not` do not work here: they require a single true-or-false answer, and an array of them raises.

```

a = np.array([0, 10, 20, 30, 40])
(a > 5) & (a < 35)    # [False  True  True  True False]
(a < 5) | (a > 35)    # [ True False False False  True]
~(a > 15)             # [ True  True False False False]

a > 5 and a < 35
# ValueError: The truth value of an array with more than one element is
# ambiguous. Use a.any() or a.all()

```

THE BRACKETS ARE NOT OPTIONAL

`&`, `|` and `~` bind more tightly than the comparison operators do. Without brackets around each comparison the expression is grouped the wrong way and either raises or quietly answers a different question.

```

a > 5 & a < 35          # parsed as  a > (5 & a) < 35
(a > 5) & (a < 35)      # what you meant

```

COUNTING AND TESTING

A boolean array answers "how many" and "is there any" directly, because `True` counts as 1:

```

mask = a > 15
mask.sum()          # 3          how many satisfy it
mask.mean()         # 0.6        what fraction
mask.any()          # True       is there at least one
mask.all()          # False      do all of them
np.count_nonzero(mask) # 3        the same count, stated explicitly

```

07

Writing through an index

Slices, index arrays and boolean masks may all appear on the left of an assignment. The right-hand side is either a single scalar or exactly as many values as were selected.

```

a = np.array([0, 10, 20, 30, 40])

a[-1] = 99          # one element
a                  # [ 0 10 20 30 99]

a[::2] = 42, 77, 11 # a slice, one value each
a                  # [42 10 77 30 11]

a[::2] = 5           # a slice, one value for all of them
a                  # [ 5 10  5 30  5]

a[[1, -1, 0]] = 7    # an index array
a[a > 6] = 0         # a boolean mask

```

Supplying the wrong number of values raises, which is the helpful case:

```

a = np.array([0, 10, 20, 30, 40])
a[::2] = 42, 99
# ValueError: could not broadcast input array from shape (2,) into shape (3,)

```

ASSIGNMENT CONVERTS TO THE ARRAY'S DTYPE

The array's dtype is fixed, so an assigned value is converted to it rather than changing it. Assigning a float into an integer array truncates, without a warning.

```

a = np.array([0, 10, 20], dtype=np.int64)
a[0] = 99.99
a                  # [99 10 20]      not 99.99

```

Because a slice is a view, writing into a slice of an array is the same as writing into the array. Writing into an index-array or mask selection is different: those produce copies when you read from them, but assigning *through* them still modifies the original, because the assignment happens before any copy is made.

```

a = np.array([0, 10, 20, 30, 40])
picked = a[[0, 1]]      # a copy
picked[0] = -1
a[0]                    # 0      the original is untouched

a[[0, 1]] = -1          # assignment through the same indices
a[0]                    # -1     the original changed

```

08

Finding the first match

Converting a boolean mask into positions, and the failure mode when no element satisfies the condition.

`np.nonzero`, and its more readable alias `np.flatnonzero`, give every position where the mask is `True`:

```

a = np.array([0, 10, 20, 30, 40])
mask = a > 15
np.flatnonzero(mask)    # [2 3 4]      every matching position
np.nonzero(mask)        # (array([2, 3, 4]),) a TUPLE, one entry per axis
np.nonzero(mask)[0]     # [2 3 4]      which is why the [0] is there

```

For the *first* match only, `argmax` on the mask is the usual approach, and it is efficient: on a boolean array it stops at the first `True` rather than scanning to the end.

```

np.argmax(a > 15)       # 2      the first position above 15

```

ARGMAX ON A COMPARISON, AND THE ANSWER IT INVENTS

When nothing matches, every element is `False`, the first of them wins, and the answer is `0`. Not `-1`, and not an exception: a valid-looking index pointing at the first element.

```

a = np.array([0, 10, 20, 30, 40])
np.argmax(a > 15)       # 2      correct
np.argmax(a > 999)       # 0      there is nothing above 999 anywhere

```

Check before you trust it:

```

hit = a > 999
idx = int(np.argmax(hit)) if hit.any() else -1

```

The same applies to `np.flatnonzero`, which returns an empty array rather than raising, so `np.flatnonzero(hit)[0]` raises `IndexError` on no match.

CHOOSING BETWEEN TWO ARRAYS

`np.where` with three arguments selects elementwise from two sources according to a mask. It returns a new array and does not modify either input:

```

a = np.array([0, 10, 20, 30, 40])
np.where(a > 15, a, 0)    # [ 0  0 20 30 40]   keep or zero
np.where(a > 15, 1, -1)   # [-1 -1  1  1  1]   two scalars

```

With one argument, `np.where(mask)` is the same as `np.nonzero(mask)` and returns a tuple of positions.

09

Loop to expression

Common loop forms and their array equivalents. Each expression in the right-hand column runs in compiled code and references no index variable.

WHAT THE LOOP DOES	AS AN ARRAY EXPRESSION
Add a constant to every element	<code>arr + 42</code>
Add two equal-length sequences	<code>a + b</code>
Running total	<code>np.cumsum(arr)</code>
Total of everything	<code>arr.sum()</code>
Count the elements passing a test	<code>(arr > 2).sum()</code>
Fraction passing a test	<code>(arr > 2).mean()</code>
Is anything true, is everything true	<code>mask.any()</code> , <code>mask.all()</code>
Largest value, and where it is	<code>arr.max()</code> , <code>arr.argmax()</code>
Keep only the elements passing a test	<code>arr[arr > 2]</code>
Positions of everything passing a test	<code>np.flatnonzero(arr > 2)</code>
Position of the first element passing a test	<code>np.argmax(arr > 2)</code> , guarded
Clamp negatives to zero	<code>np.maximum(arr, 0)</code>
Replace the elements passing a test	<code>arr[arr > 2] = 0</code>
Differences between neighbours	<code>arr[1:] - arr[:-1]</code>
Total of a stretch	<code>arr[i0:i1].sum()</code>

The last two do not announce themselves. A difference between successive elements is a subtraction of two slices offset by one, with no loop and no index arithmetic. A total over a stretch is one slice and then a reduction.

VIEW OR COPY

Getting this wrong produces either an unintended modification or an unnecessary allocation.

EXPRESSION	RESULT	WRITING INTO IT
<code>arr[2:7]</code> , <code>arr[:,2]</code>	view, no data copied	changes the original
<code>arr[[0, 2]]</code> , <code>arr[mask]</code>	a new array	independent
<code>arr.copy()</code> , <code>arr.astype(...)</code>	a new array	independent
<code>arr + 1</code> , <code>arr * 2</code> , <code>arr > 0</code>	a new array	independent
<code>arr.sum()</code> , <code>arr.max()</code>	a scalar	nothing to write into

The rule behind the table: selecting a *regular* region gives a view, while selecting an arbitrary set of positions, or computing new values, gives a new array. Elementwise arithmetic and comparison always allocate, which is why `out=` exists for the cases where that allocation is the expensive part.

FURTHER READING

- [NumPy: the absolute basics](#), which covers the ground above in the library's own words.
- [Indexing on ndarrays](#). Basic indexing, advanced indexing by index array and by boolean mask are all on this page, and it is the reference for section 5 through section 7.
- [Universal functions](#), the reference for what the operators actually call, including `out=` and the type rules.
- [The routine reference](#). Most array code people write by hand already exists there under a name.

SUMMARY

An array is one block of values of one type, fixed in both respects the moment it is made. Operators apply to every element in compiled code, which is a large constant factor and not a better algorithm. Reductions collapse an array to a single number. Slicing copies nothing, so a write through one name is visible through the other, while selecting by index array or by mask copies. And `argmax` on a comparison returns `0` when nothing matches, so it needs a guard.

Not in this document. Reducing along a named axis with `axis=`, reshaping, length-one axes, broadcasting between arrays of different shapes, the matrix product, and converting a flat index back into per-axis coordinates. All of those need more than one axis and are week 4's material.